

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:50

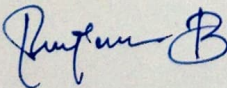
Annexure A

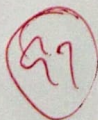
Scenario Based Assignment

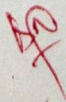
Code of Conduct:

I Poojasree B (192411091) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:







RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

20 + 20 + 25 + 20 + 15

90

ARTIFICIAL INTELLIGENCE

POOJASREE B

192411091

C8A1710

CO2-AT 2

1. Seating Arrangement

Variables

* Each guest is represented by a variable

$G_1, G_2, G_3, \dots, G_n$

* Each variable represents the table assigned to a guest

* Example: If there are 100 guests, then there are 100 variables $G_1, G_2, \dots, G_{100}$

Domains

* The domain of each variable is the set of available tables.

* If there are 10 tables,

$D(G_i) = \{T_1, T_2, T_3, \dots, T_{10}\}$

* Every guest can be assigned to any tables unless restricted by specific constraints.

Constraints

1. Conflict constraint

Guest having personal conflicts cannot sit at the same table.

Example: If Guest A conflicts with Guest B

$$\text{Table}(A) \neq \text{Table}(B)$$

2. Table Capacity constraint

Each table has a limited number of seats.

Example: If Table 1 has 8 seats,

$$\text{Guests}(T_i) \leq 8$$

3. Family Group Constraint

Members of the same family must be seated together.

Example: If Guests A, B and C belong to one family,

$$\text{Table}(A) = \text{Table}(B) = \text{Table}(C)$$

4. Accessibility / Dietary constraint

Guest with accessibility or dietary requirements must be assigned only to suitable tables.

Example: Guest D can sit only at
Tables 2 or 5. $\text{Table}(D) \in \{T_2, T_5\}$

5. VIP Constraint

VIP guests must be seated near the head table.

Example: If Tables 1 and 2 are nearest,

$\text{Table}(\text{VIP}) \in \{T_1, T_2\}$

b. Complete Assignment Constraint

Every guest must be assigned to exactly one table

$\text{Table}(G_i) = T_j$

where T_j is one valid table.

ii) Backtracking Search in this Scenario

Backtracking is a depth-first search technique that assigns guests to tables one at a time.

Working:

1. Start with the first guest.
2. Assign the guest to the first available table.
3. Check whether all constraints are satisfied.
4. If valid, move to the next guest.

5. Continue assigning guests until all guests are seated.

6. If any assignment violates a constraint undo the last assignment.

7. Try another table for the guest.

8. Repeat until a complete valid seating arrangement is found.

Example:

Assign Guest 1 $\rightarrow$ Table 1

Assign Guest 2 $\rightarrow$ Table 1

Assign Guest 3 $\rightarrow$ Table 1 (conflict)

Backtrack

Assign Guest 3 $\rightarrow$ Table 2

Continue until all guests are assigned.

iii) Constraint propagation (Forward Checking)

Forward checking improves efficiency by eliminating invalid table choices immediately after each assignment.

Working:

1. Assign a guest to a table

2. Remove that table from the domains of conflicting guests.

3. Update remaining table capacities.

4. Restrict domain based on VIP, family, and accessibility requirements

5. If any guests has no valid table remaining, backtrack immediately.

6. Continue with the reduced search space

Advantages.

- * Detects dead ends early
- * Reduces unnecessary search.
- * Decreases execution time.
- * Minimizes backtracking.
- * Improves scalability for large events.

iv) Real-world Challenges and Best Strategy for Scalability

Real-World challenges

- * Large no of guests and tables
- * Multiple overlapping constraints.
- * Last-minute guest addition or cancellation
- * Changes in VIP seating.

- * Uneven table capacities.
- * Conflicting family and friendship preferences.
- * Accessibility and dietary requirements.

Best Strategy for Scalability

The most effective approach is to combine:

- * Backtracking Search for exploring seating assignments.

- * Constraint propagation to eliminate invalid choices early.

- * Heuristics such as:

- i) Minimum Remaining Values (MRV)

- ii) Degree Heuristic:

- iii) Least Constraining Value (LCV)

2. AI Auction Bidding Game.

i) Explain how the Minimax algorithm models the problem

The Minimax algorithm is a decision making algorithm used in two-player adversarial games. In this auction-bidding scenario, the AI agent aims to maximize its

profit, while competing bidders try to maximize their own profits, which indirectly minimizes the AI's success.

Modeling the problem

Players

MAX player: AI bidding agent.

MIN player: Opponent bidding agents.

Game state

Current auction status, available budget, current highest bid, item value, and remaining rounds.

Actions

- * Place a higher bid.
- * Maintain the current bid
- * Withdraw from bidding.

Terminal State

Auction ends when no higher bids are placed.

Utility Function

- * Profit earned
- * Winning probability
- * Budget remaining
- * Overall bidding efficiency.

Working:

1. The AI assumes it is the MAX player.
2. It generates all possible bidding moves.
3. It predicts all possible responses from opponents (MIN player).
4. The process continues until a terminal state or search depth is reached.
5. The AI chooses the bid with the maximum guaranteed utility, assuming opponents play optimally.

ii) Analyze the computational challenges of large game (decision trees)

As the no of bidding rounds and possible bids increases, the game tree grows exponentially.

Challenges

1. Large Search Space
2. High Time Complexity
3. Memory Consumption
4. Strict Time Limits
5. Dynamic Market Conditions

iii) Alpha-Beta Pruning

Alpha-Beta pruning is an optimization technique for Minimax. It eliminates branches that cannot influence the final decision.

Alpha Value (α)

Best value found so far for the MAX player.

Beta Value (β)

Best value found so far for the MIN player.

Working

1. Start Minimax search
2. Update α whenever MAX finds a better value
3. Update β whenever MIN finds a lower value
4. If $\alpha \geq \beta$ stop exploring that branch.
5. Continue searching only promising branches.

iv) Design an evaluation function & justify

When the search cannot reach the end of the game, the AI estimates the desirability of a state using an evaluation function.

$$\text{Score} = \text{profit} + \text{winning probability} + \text{Budget Remaining} - \text{Risk}$$

Factors considered

1. Expected profit
2. Winning probability
3. Budget Remaining
4. Risk level
5. Market Competition.

Justification

These factors help the AI balance profitability, competitiveness, and long-term resource management instead of simply winning every auction.

V) Recommend a strategy

Combine the following techniques:

- * Minimax
- * Alpha-Beta Pruning
- * Depth-limited Search
- * Heuristic Evaluation Function
- * Iterative Deepening

i) Greedy BFS

Greedy Breadth-First Search selects the path with the lowest heuristic value ($h(n)$), i.e., the estimated distance to the survivor.

Strengths

- * Very fast
- * Uses less memory
- * Suitable for urgent rescue situations

Weaknesses

- * Ignores actual travel cost.
- * May choose unsafe or costly routes.
- * Not guaranteed to find the optimal path.

ii) A* Search

A* uses both actual cost and heuristic

$$f(n) = g(n) + h(n)$$

$g(n)$: Actual travel cost

$h(n)$: Estimated distance to the survivor

Advantages

- * Finds the optimal path
- * Considers changing movement costs and obstacles
- * Better suited for dynamic underwater environments.

iii) Impact of Heuristic Accuracy

- * Accurate heuristic
- * Poor heuristic
- * GBFS depends on heuristic Accuracy
- * A^* is less affected because it also considers actual path cost

iv) Effect of Dynamic Changes

- * Shifting debris may block planned routes
- * Changing visibility affects navigation
- * Variable currents increase travel cost
- * The AUV must update and recalculate its path using new sonar information.

v) Recommended Algorithm

A^* is the most suitable algorithm because it:

- * Balances travel cost & heuristic information.
- * Finds safer and near-optimal routes
- * Adapts better to dynamic underwaters conditions
- * Supports real-time replanning while maintaining safety & efficiency.